

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	L. Sanjay Kumar	Reg. No.:	192425226
Section / Batch:	SLOT-D/2024	Date:	15-06-2026

Code of Conduct

*I **L. Sanjay Kumar (192425226)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: L.S.Kumar

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Resume Information Extraction using Regular Expressions

Aim

To develop a Python application using Regular Expressions (Regex) that extracts important details from a resume and evaluates whether the applicant meets the required eligibility criteria.

Algorithm

1. Import the re module.
2. Read the resume information.
3. Extract the candidate's name, email, mobile number, skills, and experience.
4. Store the extracted information.
5. Verify whether the candidate has at least 2 years of experience and Python as a skill.
6. Display the candidate summary and eligibility status.

Python Programming

```
import re
resume = input("Enter Resume Details:\n")

name = re.search(r'Name\s*:\s*([A-Za-z ]+)', resume)
name = name.group(1) if name else "Not Found"

email = re.findall(r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}', resume)

mobile = re.findall(r'\b[6-9]\d{9}\b', resume)

skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills_found = []

for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        skills_found.append(skill)

exp = re.search(r'(\d+)\s+years?', resume, re.IGNORECASE)
experience = int(exp.group(1)) if exp else 0

print("\n----- Candidate Profile -----")
print("Name:", name)
print("Email:", email if email else "Not Found")
print("Mobile:", mobile if mobile else "Not Found")
print("Skills:", skills_found if skills_found else "Not Found")
print("Experience:", experience, "Years")

if experience >= 2 and "Python" in skills_found:
    print("\nCandidate Status: Eligible for Shortlisting")
else:
```

```
print("\nCandidate Status: Not Eligible")
```

Regular Expressions Used

Field	Regular Expression
Name	Name:\s*(.*)
Email	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
Mobile	\b\d{10}\b
Experience	(\d+)\s+years

Sample Input

Enter Resume Details:

Name: Rahul Sharma

Email: rahul123@gmail.com

Mobile: 9876543210

Skills: Python, Java, SQL, Machine Learning

Experience: 3 years

Sample Output

----- Candidate Profile -----

Name: Rahul Sharma

Email: ['rahul123@gmail.com']

Mobile: ['9876543210']

Skills: ['Python', 'Java', 'SQL', 'Machine Learning']

Experience: 3 Years

Candidate Status: Eligible for Shortlisting

Test Cases

<i>Name</i>	<i>Experience</i>	<i>Python Skill</i>	<i>Status</i>
<i>Rahul Sharma</i>	<i>3 Years</i>	<i>Yes</i>	<i>Eligible</i>
<i>Kiran Kumar</i>	<i>5 Years</i>	<i>No</i>	<i>Not Eligible</i>
<i>Shiva Kumar</i>	<i>1 Year</i>	<i>Yes</i>	<i>Not Eligible</i>
<i>Arun Raj</i>	<i>0 Year</i>	<i>No</i>	<i>Not Eligible</i>

Conclusion

The Python program successfully extracts resume details using Regular Expressions and determines candidate eligibility based on predefined recruitment requirements.

2. Product Search System using Regular Expressions

Aim

To create a Python-based product search system that uses Regular Expressions to perform different types of product searches efficiently.

Algorithm

1. Create a list of product names.
2. Read a search keyword from the user.
3. Perform exact, prefix, suffix, partial, and case-insensitive searches.
4. Display all matching products.
5. Generate a report showing the total number of matches.

Python Programming

```
import re
```

```
products = [  
    "Laptop",  
    "Laptop Bag",  
    "Wireless Mouse",  
    "Gaming Keyboard",  
    "USB Cable",  
    "Phone Charger",  
    "Smart Watch",  
    "Bluetooth Speaker"  
]
```

```
keyword = input("Enter Search Keyword: ")
```

```
print("\nExact Match")  
for p in products:  
    if re.fullmatch(keyword, p, re.IGNORECASE):  
        print(p)
```

```
print("\nPrefix Match")  
for p in products:  
    if re.match(keyword, p, re.IGNORECASE):  
        print(p)
```

```
print("\nSuffix Match")  
for p in products:  
    if re.search(keyword + r"$", p, re.IGNORECASE):  
        print(p)
```

```
print("\nPartial Match")  
count = 0  
for p in products:
```

```
if re.search(keyword, p, re.IGNORECASE):
```

```
    print(p)
```

```
    count += 1
```

```
print("\nTotal Matches:", count)
```

Regular Expressions Used

Search Type	Method
Exact Search	re.fullmatch()
Prefix Search	re.match()
Suffix Search	keyword\$
Partial Search	re.search()
Ignore Case	re.IGNORECASE

Sample Input

Enter Search Keyword: Laptop

Sample Output

Exact Match

Laptop

Prefix Match

Laptop

Laptop Bag

Suffix Match

Laptop

Partial Match

Laptop

Laptop Bag

Total Matches: 2

Conclusion

The Python application successfully performs multiple Regex-based search operations and displays accurate search results for different matching techniques.

3. University Registration Validation using Regular Expressions

Aim

To develop a Python application that validates university registration details using Regular Expressions and confirms successful registration only when all inputs satisfy the required format.

Algorithm

1. Accept student registration details.
2. Validate the register number, institutional email, course code, semester, and mobile number.
3. Display the validation result for each field.
4. Confirm registration if all validations are successful; otherwise, reject the registration.

Python Programming

```
import re
```

```
reg = input("Register Number: ")
email = input("Institution Email: ")
course = input("Course Code: ")
semester = input("Semester: ")
mobile = input("Mobile Number: ")

reg_pattern = r"\d{2}[A-Z]{3}\d{4}"
email_pattern = r"[A-Za-z0-9._%+-]+@saveetha\.com"
course_pattern = r"[A-Z]{3}\d{3}"
semester_pattern = r"[1-8]"
mobile_pattern = r"[6-9]\d{9}"

print("\nValidation Report")
print("-----")
if re.fullmatch(reg_pattern, reg):
    print("Register Number : Valid")
else:
    print("Register Number : Invalid")
if re.fullmatch(email_pattern, email):
    print("Email : Valid")
else:
    print("Email : Invalid")
if re.fullmatch(course_pattern, course):
    print("Course Code : Valid")
else:
    print("Course Code : Invalid")
if re.fullmatch(semester_pattern, semester):
    print("Semester : Valid")
else:
    print("Semester : Invalid")

if re.fullmatch(mobile_pattern, mobile):
```



```

print("Mobile : Valid")
else:
    print("Mobile : Invalid")
if (re.fullmatch(reg_pattern, reg) and
    re.fullmatch(email_pattern, email) and
    re.fullmatch(course_pattern, course) and
    re.fullmatch(semester_pattern, semester) and
    re.fullmatch(mobile_pattern, mobile)):
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")

```

Regular Expressions Used

Field	Regular Expression
Register Number	\d{2}[A-Z]{3}\d{4}
Email	[A-Za-z0-9._%+-]+@saveetha\.com
Course Code	[A-Z]{3}\d{3}
Semester	[1-8]
Mobile	[6-9]\d{9}

Sample Input

Register Number: 23AID1024

Institution Email: student@saveetha.com

Course Code: NLP301

Semester: 5

Mobile Number: 9876543210

Sample Output

Validation Report

Register Number : Valid

Email : Valid

Course Code : Valid

Semester : Valid

Mobile : Valid

Registration Successful

Test Cases

Register Number	Email	Course Code	Semester	Mobile	Result
23AID1024	student@saveetha.com	NLP301	5	9876543210	Successful
23AID102	student@saveetha.com	NLP301	5	9876543210	Failed
23AID1024	student@gmail.com	NLP301	5	9876543210	Failed
23AID1024	student@saveetha.com	NLP30	5	9876543210	Failed
23AID1024	student@saveetha.com	NLP301	9	9876543210	Failed

Conclusion

The Python program validates each registration field using Regular Expressions and ensures that only valid student information is accepted for successful university registration.